

3.6 矩阵和线性方程组

本节我们介绍一些与矩阵和线性方程组有关的经典题目的代码实现。

例3-32 【线性递推关系；Q矩阵】递推关系 (Recurrences, UVa 10870)

考虑线性递推关系 $f(n) = a_1 f(n-1) + a_2 f(n-2) + a_3 f(n-3) + \dots + a_d f(n-d)$ ，最著名的例子是Fibonacci数列： $f(1) = f(2) = 1$ ， $f(n) = f(n-1) + f(n-2)$ ，其中 $d=2$ ， $a_1 = a_2 = 1$ 。你的任务是计算 $f(n)$ 除以 m ($1 \leq d \leq 15$ ， $1 \leq n \leq 2^{31}-1$ ， $1 \leq m \leq 46\,340$) 的余数。

【代码实现】

```

// Uva 10870 Recurrences
// 刘汝佳
#include <cstring>
#include <iostream>
#include <string>
using namespace std;

const int NN = 20;
typedef long long Matrix[NN][NN]; // typedef 定义二维数组别名
typedef long long Vector[NN];

int sz, mod;
void matrix_mul(Matrix A, Matrix B, Matrix res) { // 矩阵乘法
    Matrix C;
    memset(C, 0, sizeof(C));
    for (int i = 0; i < sz; i++)
        for (int j = 0; j < sz; j++)
            for (int k = 0; k < sz; k++)
                C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % mod;
    memcpy(res, C, sizeof(C));
}

void matrix_pow(Matrix A, int n, Matrix res) { // 矩阵幂
    Matrix a, r;
    memcpy(a, A, sizeof(a));
    memset(r, 0, sizeof(r));
    for (int i = 0; i < sz; i++) r[i][i] = 1;
    while (n) {
        if (n & 1) matrix_mul(r, a, r);
        n >>= 1;
        matrix_mul(a, a, a);
    }
    memcpy(res, r, sizeof(r));
}

void transform(Vector d, Matrix A, Vector res) {
    Vector r;
    memset(r, 0, sizeof(r));
    for (int i = 0; i < sz; i++)
        for (int j = 0; j < sz; j++) r[j] = (r[j] + d[i] * A[i][j]) % mod;
    memcpy(res, r, sizeof(r));
}

int main() {
    for (int d, n, m; cin >> d >> n >> m && d;) {
        Matrix A;
        Vector a, f;
        for (int i = 0; i < d; i++) cin >> a[i], a[i] %= m;
        for (int i = d - 1; i >= 0; i--) cin >> f[i], f[i] %= m;
        memset(A, 0, sizeof(A));
        for (int i = 0; i < d; i++) A[i][0] = a[i];
        for (int i = 1; i < d; i++) A[i - 1][i] = 1;
        sz = d, mod = m;
        matrix_pow(A, n - d, A);
        transform(f, A, f);
        cout << f[0] << endl;
    }
    return 0;
}

/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 2.7 节例题 26
同类题目：233 Matrix, HDU 5015
          Queuing, HDU 2604
*/

```

例3-33 【循环矩阵的乘法】细胞自动机 (Cellular Automaton, NEERC 2006, POJ 3150)

一个细胞自动机包含 n 个格子，每个格子的取值为 $0 \sim m-1$ 。给定距离 d ，则每次操作后每个格子的值将变为到它的距离不超过 d 的所有格子在操作之前的值之和除以 m 的余数，其中 i 和 j 的距离为 $\min\{|i-j|, n-|i-j|\}$ 。给出 n, m, d, k ($1 \leq n \leq 500$, $1 \leq m \leq 10^6$, $0 \leq d < n/2$, $1 \leq k \leq 10^7$) 和自动机各格子的初始值，你的任务是计算 k 次操作以后各格子的值。

如图3-7所示， $n=5, m=3, d=1$ ，一次操作将把图3-7 (a) 变为图3-7 (b)。比如，与格子3距离不超过1的格子（即格子2,3,4）在操作前的值分别为2, 2, 1，因此操作后格子3的值为 $(2+2+1) \bmod 3=2$ 。

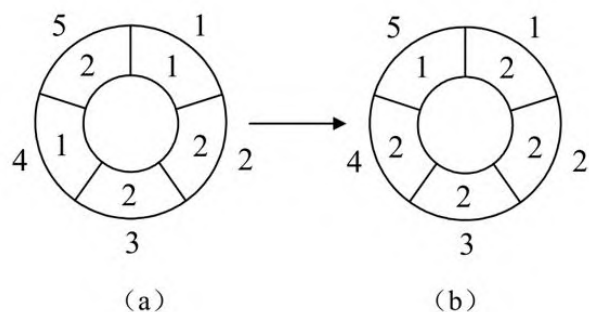


图3-7 细胞自动机

【代码实现】

```

// 刘汝佳
#include <cstdio>
#include <cstring>
#include <algorithm>
using namespace std;
typedef long long LL;
const int maxn = 500 + 8;
int MOD;
struct Matrix {
    int a[maxn], n;
    Matrix(int _n = 1) : n(_n) { fill_n(a, n + 1, 0); }
    Matrix operator * (const Matrix &rhs) { // 重载乘法运算符
        Matrix m(n);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                (m.a[i] += (LL)a[(i - j + n) % n] * rhs.a[j] % MOD) %= MOD;
        return m;
    }
};

Matrix fast_pow(Matrix x, int n) {
    Matrix m(x.n); m.a[0] = 1;
    while (n) {
        if (n % 2) m = m * x;
        x = x * x, n /= 2;
    }
    return m;
}

int main() {
    for (int d, k, n, m; scanf("%d %d %d %d", &n, &m, &d, &k) == 4;) {
        MOD = m;
        Matrix x(n), y(n);
        for (int i = 0; i < n; ++i) scanf("%d", &x.a[i]);
        fill_n(y.a, d + 1, 1), fill_n(y.a + n - d, d, 1);
        Matrix ans = x * fast_pow(y, k);
        for (int i = 0; i < n; ++i) printf("%d%c", ans.a[i], " \n"[i + 1 == n]);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 2.7 节例题 27
同类题目：Kiki & Little Kiki 2, HDU 2276
*/

```

例3-34 【高斯消元；马尔可夫过程；实数域的线性方程组（有特殊情况）】随机程序（Back to Kernighan-Ritchie, UVa 10828）

给出一个类似图3-8的程序控制流图，从每个结点出发到每个后继结点的概率均相等。当执行完一个没有后继的结点后，整个程序终止。程序总是从编号为1的结点开始执行。你的任务是对于若干个查询结点，求出每个结点的期望执行次数。

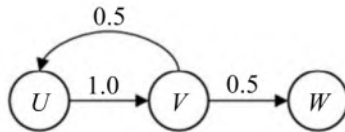


图3-8 程序控制流图

比如，在图3-8中，如果从结点 U 开始执行，那么结点 U , V 和 W 的期望执行次数分别为2, 2, 1。

输入包含不超过100组数据。每组数据：第一行为整数 n ($1 \leq n \leq 100$)，即控制流图的结点数，结点编号为 $1 \sim n$ ；以下若干行每行包含两个整数 a, b ，表明从结点 a 执行完后可以紧接着执行结点 b ，这部分以两个0结束；下一行为整数 q ($q \leq 100$)，即查询的个数；以下 q 行每行为一个结点编号。输入结束标志为 $n=0$ 。

【代码实现】

```

// 刘汝佳
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <cstring>
#include <vector>
using namespace std;

const double eps = 1e-8;
const int NN = 100 + 10;
typedef double Matrix[NN][NN];

// 由于本题的特殊性，消元后不一定是对角矩阵，甚至不一定是阶梯矩阵
// 若  $x[i]$  的解唯一且有限，第  $i$  行除  $A[i][i]$  和  $A[i][n]$  外的其他元素均为 0
void gauss_jordan(Matrix A, int n) {
    int i, j, k, r;
    for (i = 0; i < n; i++) {
        r = i;
        for (j = i + 1; j < n; j++)
            if (fabs(A[j][i]) > fabs(A[r][i])) r = j;
        if (fabs(A[r][i]) < eps) continue; // 放弃这一行，直接处理下一行 (*)
        if (r != i)
            for (j = 0; j <= n; j++) swap(A[r][j], A[i][j]);
    }
}

```

```

// 与除第 i 行外的其他行进行消元
for (k = 0; k < n; k++)
    if (k != i)
        for (j = n; j >= i; j--) A[k][j] -= A[k][i] / A[i][i] * A[i][j];
}
}

int main() {
    int d[NN], inf[NN];
    Matrix A;
    vector<int> pre[NN];
    for (int kase = 1, n; scanf("%d", &n) == 1 && n; kase++) {
        memset(d, 0, sizeof(d));
        for (int i = 0; i < n; i++) pre[i].clear();
        for (int a, b; scanf("%d%d", &a, &b) == 2 && a; pre[b].push_back(a))
            a--, b--, d[a]++; // 改成从 0 开始编号, a 的出度加 1
        // 构造方程组
        memset(A, 0, sizeof(A));
        for (int i = 0; i < n; i++) {
            A[i][i] = 1;
            for (int j = 0; j < pre[i].size(); j++)
                A[i][pre[i][j]] -= 1.0 / d[pre[i][j]];
            if (i == 0) A[i][n] = 1;
        }

        // 解方程组, 标记无穷变量
        gauss_jordan(A, n);
        memset(inf, 0, sizeof(inf));
        for (int i = n - 1; i >= 0; i--) {
            if (fabs(A[i][i]) < eps && fabs(A[i][n]) > eps)
                inf[i] = 1; // 直接解出来的无穷变量
            for (int j = i + 1; j < n; j++)
                if (fabs(A[i][j]) > eps && inf[j])
                    inf[i] = 1; // 和无穷变量有关的变量也是无穷的
        }

        int q, u;
        scanf("%d", &q);
        printf("Case %d:\n", kase);
        while (q--) {
            scanf("%d", &u), u--;
            if (inf[u])
                printf("infinity\n");
            else
                printf("%.3lf\n", fabs(A[u][u]) < eps ? 0.0 : A[u][n] / A[u][u]);
        }
    }
    return 0;
}
/*

```

算法分析请参考:《算法竞赛入门经典——训练指南》升级版 2.7 节例题 28

同类题目: Kiki & Little Kiki 2, HDU 2276

游走, HNOI 2013, 牛客 NC 20105

*/

例3-35 【XOR方程组】乘积是平方数 (Square, UVa 11542)

给出 n ($1 \leq n \leq 100$) 个整数, 所有整数均不小于1, 不大于 10^{15} , 并且不含大于500的素因子。从中选出1个或多个整数, 使得选出的整数的乘积是完全平方数。一共有多少种选法? 比如, $\{4, 6, 10, 15\}$ 有3种选法: $\{4\}, \{6, 10, 15\}, \{4, 6, 10, 15\}$ 。

【代码实现】


```

// 刘汝佳
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <cstring>
#include <iostream>
#include <vector>
using namespace std;

const int NN = 500 + 10, maxp = 100;
int vis[NN], prime[maxp];
int gen_primes(int n) {
    int m = (int)sqrt(n + 0.5);
    fill_n(vis, NN, 0);
    for (int i = 2; i <= m; i++)
        if (!vis[i])
            for (int j = i * i; j <= n; j += i) vis[j] = 1;
    int c = 0;
    for (int i = 2; i <= n; i++)
        if (!vis[i]) prime[c++] = i;
    return c;
}

typedef int Matrix[NN][NN];

// m个方程, n个变量
int get_rank(Matrix A, int m, int n) {
    int i = 0, j = 0, k, r, u;
    while (i < m && j < n) {
        r = i;
        for (k = i; k < m; k++)
            if (A[k][j]) {
                r = k;
                break;
            }
        if (A[r][j]) {
            if (r != i)
                for (k = 0; k <= n; k++) swap(A[r][k], A[i][k]);
            // 消元后第 i 行的第一个非 0 列是第 j 列, 且第 u>i 行的第 j 列均为 0
            for (u = i + 1; u < m; u++)
                if (A[u][j])
                    for (k = j + 1; k < n; k++)
                        A[u][k] -= A[i][k] * A[u][j] / A[i][j];
        }
        i++;
        j++;
    }
    return i;
}

```

```

        for (u = i + 1; u < m; u++)
            if (A[u][j])
                for (k = i; k <= n; k++) A[u][k] ^= A[i][k];
        i++;
    }
    j++;
}
return i;
}

Matrix A;

int main() {
    int m = gen_primes(500), T;
    cin >> T;
    while (T--) {
        int n, maxp = 0;
        long long x; // 注意 x 的范围
        cin >> n;
        memset(A, 0, sizeof(A));
        for (int i = 0; i < n; i++) {
            cin >> x;
            for (int j = 0; j < m; j++) // 求 x 中的 prime[j] 的幂, 并更新系数矩阵
                while (x % prime[j] == 0)
                    maxp = max(maxp, j), x /= prime[j], A[j][i] ^= 1;
        }
        int r = get_rank(A, maxp + 1, n); // 只用到了前 maxp+1 个素数
        cout << (1LL << (n - r)) - 1 << endl; // 空集不是解, 所以要减 1
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 2.7 节例题 29
同类题目: XOR, HDU 3949
            EXTENDED LIGHTS OUT, ACM/ICPC Greater NY 2012, POJ 1222
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点, 如表3-6所示。

表3-6 矩阵和线性方程组例题归纳